Introdução à Orientação a Objetos

Aula 03 — Modificadores de acesso e encapsulamento

Programação Orientada a Objetos

Semana 3

Sumário

1. Introdução
2. O problema: regra de negócio burlada
3. Modificadores de visibilidade
4. Isolando comportamentos
5. Encapsulamento
6. Getters e setters
7. Exercício de fixação
8. Encerramento

Revisão da aula anterior

- **Classe**: molde. **Objeto**: instância criada com `new`
- Atributos guardam o **estado**; métodos guardam o **comportamento**
- `this` aponta explicitamente para o atributo da classe
- Variável de objeto guarda **referência**, não cópia
- Nosso projeto: `Livro`, `Autor` e `CadastroDeLivros`

Hoje

- Um problema sério na nossa livraria: a regra de negócio pode ser **burlada**
- Modificadores de visibilidade: `private`, *default*, `protected`, `public`
- **Encapsulamento**: esconder os dados e esconder o *como*
- Getters e setters

Nova regra de negócio

Um livro pode ter no máximo **30%** de desconto.

- Nosso `aplicaDescontoDe` de ontem aceitava qualquer porcentagem
- Precisamos validar antes de alterar o valor

Validando o desconto

```
1 public boolean aplicaDescontoDe(double porcentagem) {
2     if (porcentagem > 0.3) {
3         return false;
4     }
5     this.valor -= this.valor * porcentagem;
6     return true;
7 }
```

- Retorna `boolean` indicando se o desconto foi aplicado
- `return` interrompe o método na hora

Usando o método

```
1 if (!livro.aplicaDescontoDe(0.1)) {
2     System.out.println("Desconto nao pode ser maior do que 30%");
3 }
```

- Quem chama o método decide o que fazer quando a regra barra

Por que não imprimir dentro do método?

```
1 public void aplicaDescontoDe(double porcentagem) {
2     if (porcentagem > 0.3) {
3         System.out.println("Desconto nao pode ser maior do que 30%");
4     }
5     this.valor -= this.valor * porcentagem;
6 }
```

- Parece mais prático, mas **perdemos flexibilidade**
- E se outro programa quiser outra mensagem? Ou nenhuma?
- Evite pendurar no molde responsabilidades que não são dele

Testando a regra

```java
public class RegrasDeDesconto {
    public static void main(String[] args) {
        Livro livro = new Livro();
        livro.valor = 59.90;

        System.out.println("Valor atual: " + livro.valor);

        if (!livro.aplicaDescontoDe(0.1)) {
            System.out.println("Desconto nao pode ser maior do que 30%");
        } else {
            System.out.println("Valor com desconto: " + livro.valor);
        }
    }
}
```

Saída: `Valor atual: 59.9` / `Valor com desconto: 53.91`

Mas há um problema grave

```
1 Livro livro = new Livro();
2 livro.valor = 59.90;
3
4 // ninguem obriga o programador a usar aplicaDescontoDe:
5 livro.valor -= livro.valor * 0.4;    // 40% de desconto!
```

- O atributo `valor` pode ser acessado e modificado por qualquer parte do sistema
- A regra de negócio foi **ignorada** — e o código compila normalmente
- Expor os dados = permitir que o objeto seja modificado sem passar pelas validações

A solução: restringir o acesso

```
1 public class Livro {
2     private double valor;
3     // outros atributos e metodos
4 }
```

- Até agora nossos atributos estavam com visibilidade *default* (padrão)
- Com `private`, só a própria classe `Livro` acessa `valor`

```
1 livro.valor -= livro.valor * 0.4;
2 // erro de compilacao:
3 // The field Livro.valor is not visible
```

Os quatro modificadores

`private`	só a própria classe
default (sem palavra)	classes do mesmo pacote
`protected`	mesmo pacote + subclasses
`public`	qualquer classe

- Hoje usaremos apenas `private` e `public`
- *default* e `protected` dependem de **pacotes** e **herança** — voltaremos a eles

Erro de compilação é bom!

- Tornar `valor` privado quebrou vários pontos do código
- Isso é exatamente o que queríamos: o compilador agora **obriga** todo mundo a passar pela regra de negócio
- Melhor descobrir na compilação do que com um livro vendido com 90% de desconto

Quebrou a atribuição do valor

```
1 livro.valor = 59.90;   // nao compila mais!
```

Precisamos de um **comportamento** na classe `Livro` para isso:

```
1 void adicionaValor(double valor) {
2     this.valor = valor;
3 }
```

```
1 livro.adicionaValor(59.90);
```

Repare no `this`: distingue o atributo do parâmetro de mesmo nome

Quebrou também a leitura do valor

```
System.out.println("Valor atual: " + livro.valor); // nao compila
```

Mesma ideia, agora para ler:

```
double retornaValor() {
    return this.valor;
}
```

```
System.out.println("Valor atual: " + livro.retornaValor());
```

Como o método não recebe parâmetros, aqui o `this` é opcional

E os outros atributos? Quando usar `private`?

Hoje o nome ainda é atribuído direto:

```
1 livro.nome = "Java 8 Pratico";
```

Daqui a alguns meses aparece a regra: nome precisa ter ao menos 2 letras.

```
1 livro.nome = "Java 8 Pratico";
2 if (livro.nome.length() < 2) {
3     System.out.println("Nome invalido");
4 }
```

- Se o atributo é acessado direto em mil lugares, é preciso repetir a validação em mil lugares

Só criar o método não resolve

- Criar `adicionaNome` ajuda, mas se o atributo continuar *default* existem **duas** formas de atribuir o nome
- Alguém vai usar o caminho errado — e a validação fica furada de novo
- Regra prática:

Todo atributo de classe deve ser `private`.

O conceito

- O que fizemos tem nome: **encapsulamento**
- Esconder os atributos (todos `private`)
- Encapsular os comportamentos em métodos
- Esconder **como** as regras de negócio funcionam

O termômetro: duas perguntas

O que esse código faz? *Como esse código faz?*

```
1 livro.valor -= livro.valor * 0.4;
```

Conseguimos responder **as duas** — sinal de código mal encapsulado.

```
1 livro.aplicaDescontoDe(0.1);
```

Sabemos **o que** faz, mas não **como**. Em código bem encapsulado, só a primeira pergunta tem resposta.

Por que isso importa

- Amanhã queremos enviar um e-mail sempre que um desconto for aplicado
- Código encapsulado: muda **um** método
- Código exposto: caçar milhares de pontos espalhados pelo sistema

Encapsulamento deixa o código muito mais **passível de mudanças**.

Interface da classe

- Os métodos públicos de uma classe são chamados de **interface da classe**
- Em código encapsulado, são a **única** forma de interagir com os objetos
- Analogia: o teclado é a interface do computador
  - Você digita sem saber como os sinais são interpretados
  - Trocar de teclado não exige reaprender a escrever
  - Ninguém abre a lateral da máquina e mexe nos circuitos

A convenção get / set

```java
public class Livro {
    private double valor;

    public double getValor() {
        return valor;
    }

    public void setValor(double valor) {
        this.valor = valor;
    }
}
```

- get: sem parâmetro, retorna o atributo
- set: recebe um parâmetro do mesmo tipo do atributo, não retorna nada

Nomenclatura

Atributo	Métodos
`nome`	`getNome()` / `setNome(String nome)`
`valor`	`getValor()` / `setValor(double valor)`
`autor`	`getAutor()` / `setAutor(Autor autor)`

- Prefixo + nome do atributo em `CamelCase`
- É apenas uma convenção — mas seguida por toda a comunidade Java

Gerando pela IDE

- Escrever getters e setters à mão é repetitivo — as IDEs geram por você
- **IntelliJ IDEA**: menu *Code* → *Generate...* (`Alt + Insert`) → *Getter and Setter*
- **Eclipse**: `Ctrl + 3` e digitar ggas (*Generate Getters and Setters*)
- Selecione os atributos desejados e confirme

A classe Livro encapsulada

```java
public class Livro {
    private String nome;
    private String descricao;
    private double valor;
    private String isbn;
    private Autor autor;

    // demais metodos omitidos

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public double getValor() { return valor; }
    public void setValor(double valor) { this.valor = valor; }
}
```

A classe Autor encapsulada

```java
public class Autor {
    private String nome;
    private String email;
    private String cpf;

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getCpf() { return cpf; }
    public void setCpf(String cpf) { this.cpf = cpf; }
}
```

O CadastroDeLivros adaptado

```java
public class CadastroDeLivros {
    public static void main(String[] args) {
        Autor autor = new Autor();
        autor.setNome("Rodrigo Turini");
        autor.setEmail("rodrigo.turini@caelum.com.br");

        Livro livro = new Livro();
        livro.setNome("Java 8 Pratico");
        livro.setDescricao("Novos recursos da linguagem");
        livro.setValor(59.90);
        livro.setIsbn("978-85-66250-46-6");
        livro.setAutor(autor);

        livro.mostrarDetalhes();
    }
}
```

Cuidado: get/set não é obrigação

- Getters e setters dão acesso **controlado** — mas só crie os que forem necessários
- Gerar get/set para todos os atributos é voltar a expor tudo, só que com mais linhas

```
1 // faz sentido?
2 public void setIdade(int idade) { this.idade = idade; }
3
4 // ou a regra de negocio real e:
5 public void fazAniversario() { this.idade++; }
```

blog.caelum.com.br/2006/09/14/nao-aprender-oo-getters-e-setters/

Setter pode validar

```java
public void setNome(String nome) {
    if (nome.length() < 2) {
        System.out.println("Nome invalido");
        return;
    }
    this.nome = nome;
}
```

- A validação fica em **um** lugar, e ninguém consegue escapar dela
- É exatamente o problema do início da aula, agora resolvido

Encapsule a classe `Produto`

```java
public class Produto {
    String nome;
    double preco;
    int quantidadeEmEstoque;
}
```

1. Torne todos os atributos `private`
2. Crie os getters e setters necessários
3. Crie o método `retiraDoEstoque(int quantidade)`, que só retira se houver estoque suficiente e retorna `boolean`
4. Garanta que não exista nenhuma outra forma de diminuir o estoque

Uma solução possível

```java
public class Produto {
    private String nome;
    private double preco;
    private int quantidadeEmEstoque;

    public boolean retiraDoEstoque(int quantidade) {
        if (quantidade > this.quantidadeEmEstoque) {
            return false;
        }
        this.quantidadeEmEstoque -= quantidade;
        return true;
    }

    public int getQuantidadeEmEstoque() { return quantidadeEmEstoque; }
}
```

Note: **não** existe `setQuantidadeEmEstoque` — de propósito.

Resumo da aula

- Atributo exposto = regra de negócio que pode ser burlada
- Modificadores: `private`, *default*, `protected`, `public`
- **Todo atributo de classe deve ser** `private`
- Encapsulamento: esconder os dados e esconder o *como*
- Termômetro: em código bem encapsulado, só dá pra responder *o que* o código faz
- Getters e setters — criados sob demanda, não por reflexo

Próxima aula

Construtores, sobrecarga de métodos e de construtores

Tarefa: resolver a lista de exercícios da aula 03